

Names: Lesedi Shelile, Kiara Ajodhaparsadh, Ajba Patel

Task 1

GameVerse Convention

GameVerse Convention is a large-scale gaming convention where gamers, developers, esports teams, and content creators gather to participate in tournaments, experience new games, explore virtual reality demonstrations, and purchase gaming merchandise. The convention is coordinated through a central **Event Control** system that manages changes to different areas and keeps operational units informed throughout the event.

The convention is divided into several major areas. **Main Operations** manages attendee registration, information, and security. **Esports Hall** hosts competitive gaming tournaments and practice areas. **VR & Experience Hall** provides virtual-reality experiences and game demonstrations. **VIP & Creator Lounge** contains VIP Services and a Creator Area, providing dedicated services for VIP guests and content creators.

The event is represented using a **Composite** structure in which larger areas contain smaller areas and individual operational units. Examples of concrete event units (leaves) include **Check-In Desk, Badge Station, Information Desk, Security Station, Entrance Gate, Security Team, First Aid Station, Medical Team, Esports Stage, Referee Team, Spectator Gate, Gaming Station, Technical Support, VR Station, Safety Team, Game Demo Booth, VIP Check-In Desk, VIP Support, Creator Support, and Media Station.**

These units have different responsibilities and can react differently to event notifications. For example, a **Security Team** can respond to a safety alert by securing an area, while an **Esports Stage** can respond to the same alert by pausing a tournament.

Examples of grouping/area (**Composite**) types include **Main Operations, Registration Centre, Security Operations, Medical Services, Esports Hall, Championship Arena, Practice Area, VR & Experience Hall, VR Zone, Demo Zone, VIP & Creator Lounge, VIP Services, and Creator Area.**

These groups can contain other groups as well as individual event units, allowing the entire convention to be treated as one event while still allowing individual areas and units to be managed separately.

During the convention, **Event Control** can issue notifications such as opening or closing an area, schedule changes, capacity alerts, server outages, safety alerts, weather alerts, temporary pauses, resume instructions, and evacuation instructions. These notifications allow the different areas and event units to coordinate as conditions change.

The design therefore uses **Composite** to represent what is contained within the convention and its areas, while **Observer** is used to communicate important changes to the parts of the event that need to respond. This follows the practical's distinction between Composite as the **part-whole structure** and Observer as the **mechanism for communicating changes**.

1.2

Composite

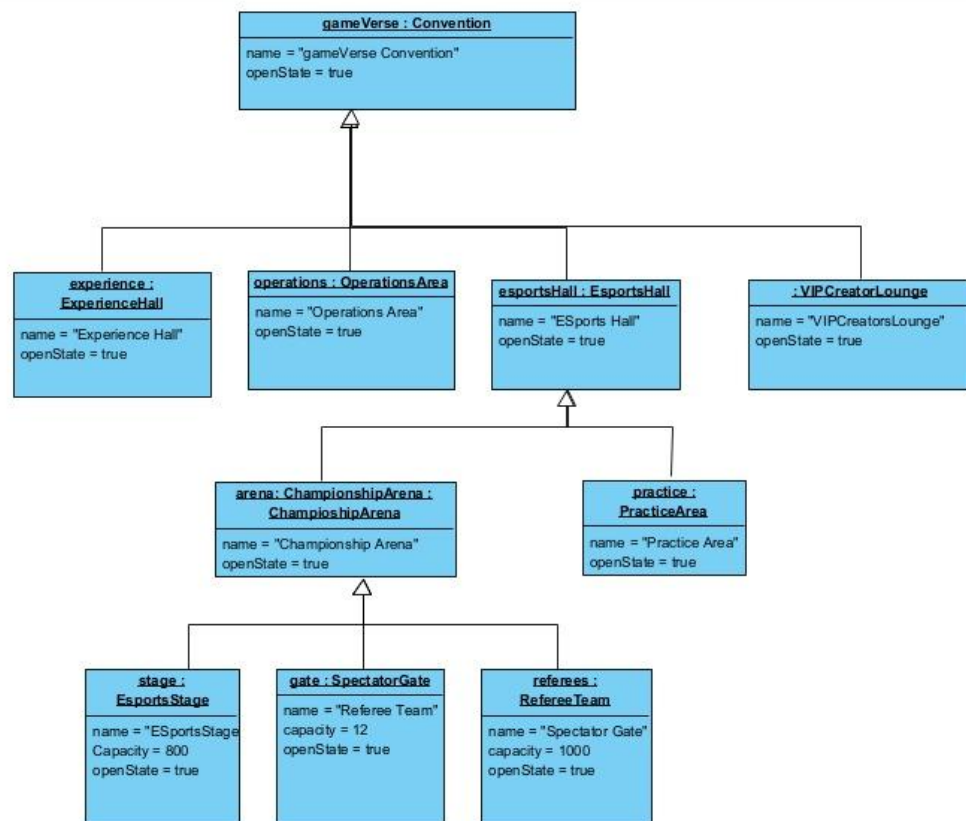
GoF Participant	GameVerse class	Role
Component	EventComponent	Common interface for all event elements
Leaf	EventUnit and its concrete subclasses	Represents individual operational units
Composite	EventGroup	Represents areas that contain other components
Client	EventControl / main.cpp	Builds and operates on the event tree

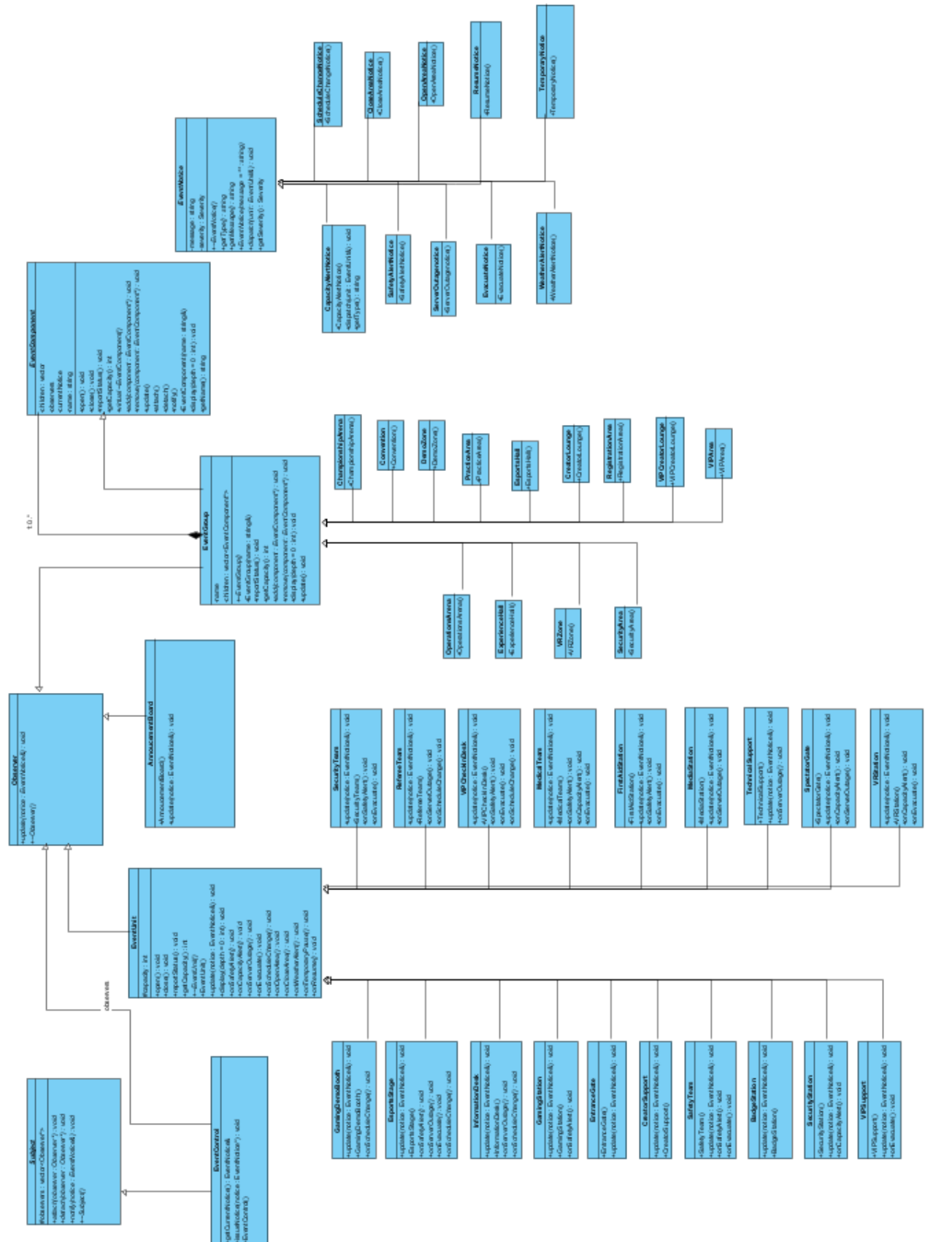
Observer

GoF Participant	className	Role
Subject	Subject	Interface for attaching, detaching and notifying observers
ConcreteSubject	EventControl	Holds the current event notice/state and notifies observers
Observer	Observer	Interface defining update(...)
ConcreteObserver	EventGroup and selected EventUnit classes	React to event notifications

EventGroup has two GFO roles, as a composite it contains championshiparena and practicearea and as an observer it receives notifications from eventcontrol and as a subject After receiving the notification, it can notify the interested objects inside the hall. This is intentional because its Composite role manages containment while its Observer/Subject roles manage event communication.

1.3





1.4 Design Rationale

a)

The Composite pattern represents GameVerse as a genuine part-whole tree. The convention is the root EventGroup and contains major areas such as Main Operations, Esports Hall, VR & Experience Hall and VIP & Creator Lounge. These groups contain smaller groups, which in turn contain individual EventUnit leaves. Because EventGroup and EventUnit share EventComponent, the client can operate on an individual unit or an entire area through the same interface. EventGroup owns its child EventComponent objects and recursively destroys the owned subtree

b)

The Observer pattern separates event changes from the objects that must react to them. EventControl does not make hard-coded calls to every concrete EventUnit. Instead, observers register with a Subject and receive notices through update(const EventNotice&). This means new observers can be added without modifying EventControl. Different observers can respond differently to the same notice through their own polymorphic behaviour.

c)

EventGroup has multiple pattern roles intentionally. As a Composite it manages the structure of an event area. As an Observer it receives a notice from the level above. As a Subject it forwards the notice to interested objects below. These are separate collaborations, so the class is not a misuse of the patterns. This supports cascading notifications through the event hierarchy.

d)

EventGroup owns its EventComponent children. Its destructor deletes each owned child exactly once. remove() and takeChild() detach the component before transferring structural ownership. Subject observer

pointers are non-owning, so EventControl and EventGroup never delete Observer pointers.

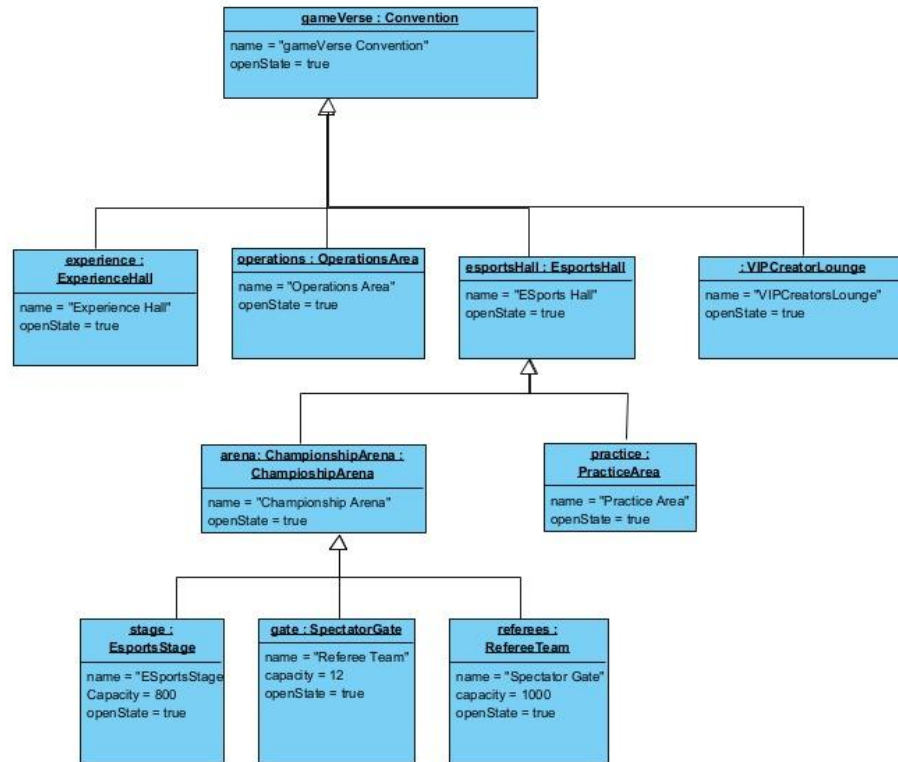
e)

In this implementation, EventGroup::add()

intentionally couples ownership registration with observation registration so that the Composite hierarchy and notification hierarchy remain aligned. This is a deliberate design decision and is reflected in the code

Task 2

2.3



2.4

Rule 1 — Safety Alert

When a **SAFETY_ALERT** is issued, each unit responds according to its responsibility:

- **Esports Stage** → pauses the current tournament match.
- **Gaming Station** → stops accepting players and saves the current session.
- **Security Team** → secures the affected area and controls attendee movement.
- **Medical Team** → remains operational and prepares to assist attendees.
- **VIP Check-In Desk** → temporarily restricts VIP access.

The controller does **not** need to know what each unit is. Each concrete class implements its own `update()` behaviour.

Rule 2 — Capacity Alert

When a **CAPACITY_ALERT** is issued:

- **Spectator Gate** → stops admitting additional spectators.

- **Check-In Desk** → temporarily stops registering new attendees.
- **Security Station** → sends security personnel to manage the queue.
- **VR Station** → stops accepting new users but allows the current session to finish.
- **Medical Team** → remains operational because medical capacity is not affected by attendee admission.

This demonstrates that the same notification can cause different actions depending on the object's state and responsibility.

Rule 3 — Server Outage

When a **SERVER_OUTAGE** occurs in the Esports Hall:

- **Esports Stage** → pauses the tournament.
- **Referee Team** → suspends the current match and records its status.
- **Technical Support** → investigates and attempts to restore the server.
- **Spectator Gate** → remains open unless the resulting crowd reaches the capacity threshold.
- **Information Desk** → informs attendees that the tournament has been delayed.

This is particularly useful for our sequence diagrams because several objects can react to the same event in different ways.

Rule 4 — Evacuation

When an **EVACUATE** notice is issued:

- **Security Team** → directs attendees toward designated exits.
- **Entrance Gate** → switches to emergency-exit mode and permits people to leave.
- **Esports Stage** → immediately pauses the tournament.
- **VR Station** → safely shuts down the VR equipment.
- **Medical Team** → remains active and prepares for potential injuries.
- **VIP Check-In Desk** → stops admitting VIP guests and directs them toward the evacuation route.

The different responses are determined by each object's own behaviour rather than a large if/else statement in EventControl.

Rule 5 — Schedule Change

When a **SCHEDULE_CHANGE** notice is issued:

- **Esports Stage** → updates the tournament start time.
- **Referee Team** → updates its match schedule.
- **Information Desk** → updates the information available to attendees.
- **VIP Check-In Desk** → updates VIP guests about affected activities.
- **Game Demo Booth** → updates its demonstration timetable.

For example, if the championship is delayed by 30 minutes, the stage and referee team need to change their schedules, while Security does not need to change its behaviour

TASK 3

3.1 Subject

Subject defines the common Observer registration operations `attach(Observer*)`, `detach(Observer*)` and `notify(const EventNotice&)`. The registration list is stored as `std::vector` and is non-owning. A null observer is ignored. Duplicate attachment is ignored, so an observer appears at most once in a Subject's list. Detaching an observer that is not registered has no effect. `notify()` first takes a snapshot of the current observer list and then calls `update(notice)` on the snapshot. Consequently, if an observer changes registration during notification, that change does not invalidate the current iteration; it affects the next notification. This gives the design a clear runtime-registration policy

3.2 Observer

Observer is an abstract interface containing `update(const EventNotice&)`. `EventUnit` implements Observer and delegates the notice to the polymorphic `EventNotice::dispatch(EventUnit&)` operation. Concrete event units then implement responsibility-specific handlers such as `onSafetyAlert()`, `onServerOutage()`, `onEvacuate()` and `onScheduleChange()`. `EventGroup` also implements Observer; its `update()` receives a notice and calls `notify(notice)`, allowing the notice to cascade further down the Composite.

3.3 Notice/Order Types

The six core types are `SafetyAlertNotice`, `CapacityAlertNotice`, `ServerOutageNotice`, `EvacuateNotice`, `ScheduleChangeNotice` and `OpenAreaNotice`. Additional types are `CloseAreaNotice`, `WeatherAlertNotice`, `TemporaryPauseNotice` and `ResumeNotice`.

Each `EventNotice` also carries `NoticeSeverity`: `LOW`, `MEDIUM` or `HIGH`. The notice exposes `getSeverity()`, `getSeverityName()` and `getPublicMessage()`, while concrete notices retain polymorphic dispatch to the appropriate `EventUnit` behaviour.

3.4 Cascading Notifications

Cascading is implemented through `EventGroup`'s dual `Observer/Subject` role. `EventControl` pushes a notice to the root `Convention`. `Convention` receives it as an `Observer` and calls `notify()`, which forwards the same notice to interested descendants. Nested `EventGroups` repeat this process until concrete `EventUnits` receive the notice

EventControl → Convention → Esports Hall Championship Arena → Esports Stage / Referee Team / Spectator Gate

3.5 PUSH MODEL

The `Subject` sends the relevant `EventNotice` directly to `Observer::update(const EventNotice&)`. The pushed notice contains the message, severity and concrete notice behaviour needed by the observer. The observer therefore does not need to query the `Subject` for the current notice.

Push was selected because `GameVerse` notices already represent the information that observers need. It makes the notification chain explicit, allows the same notice to cascade through `EventGroups`, and lets observers interpret the notice independently. The trade-off is that the notification payload must contain the information observers may require.

TASK 4

4.3

The primary new condition uses notice severity. `EventNotice` carries `LOW`, `MEDIUM` or `HIGH`. `EsportsStage` checks the severity it receives. When a `HIGH`-severity notice arrives, the stage performs an immediate-attention response before executing the normal notice-specific polymorphic action. Lower-severity notices follow the normal notice-specific response

4.4 ORIGINAL FEATURES

1. Notice Severity

Every EventNotice carries LOW, MEDIUM or HIGH severity. Severity allows an observer to distinguish routine, warning and urgent conditions. For example, a HIGH ServerOutageNotice causes EsportsStage to perform an immediate-attention response before pausing the tournament.

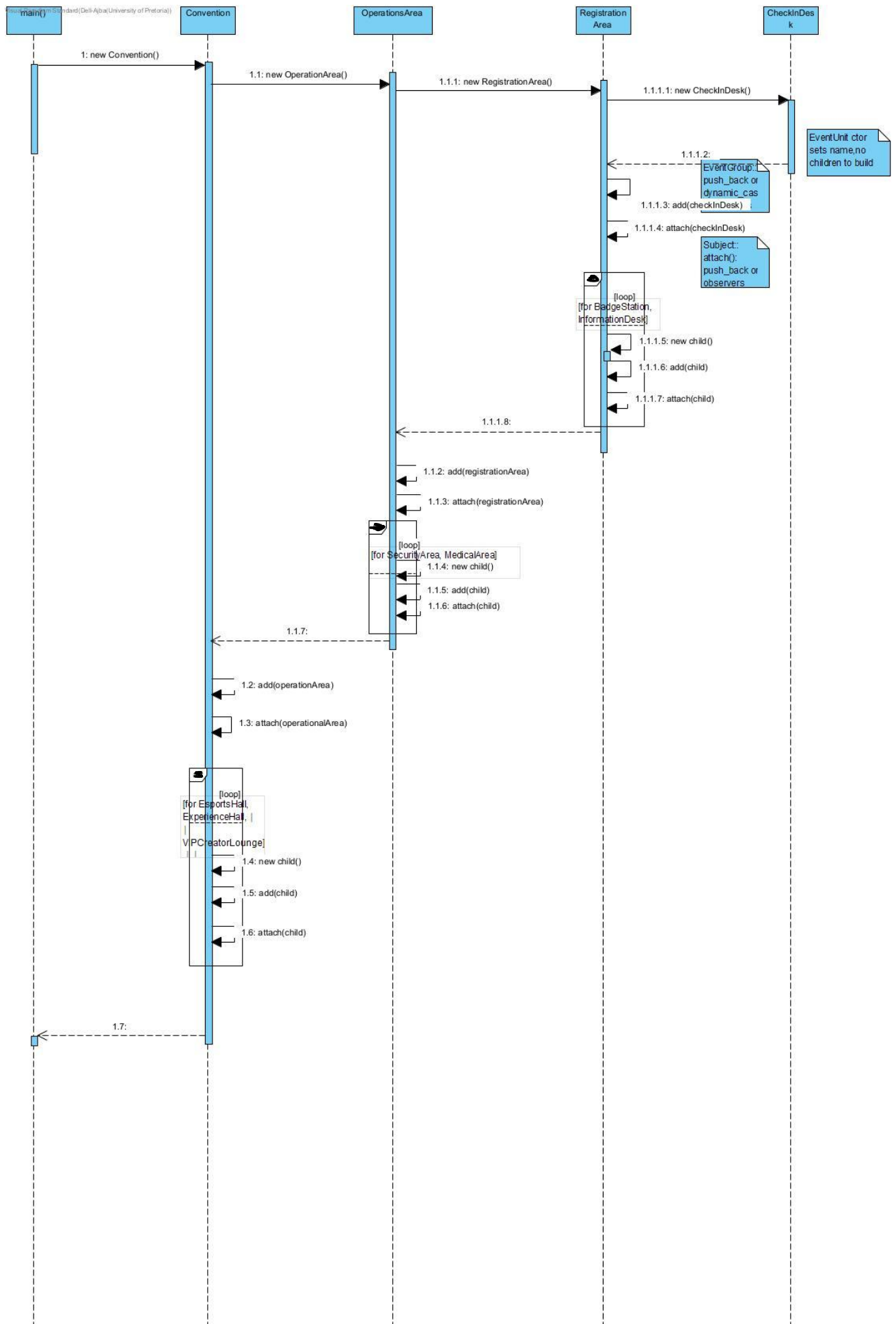
2. Announcement Board

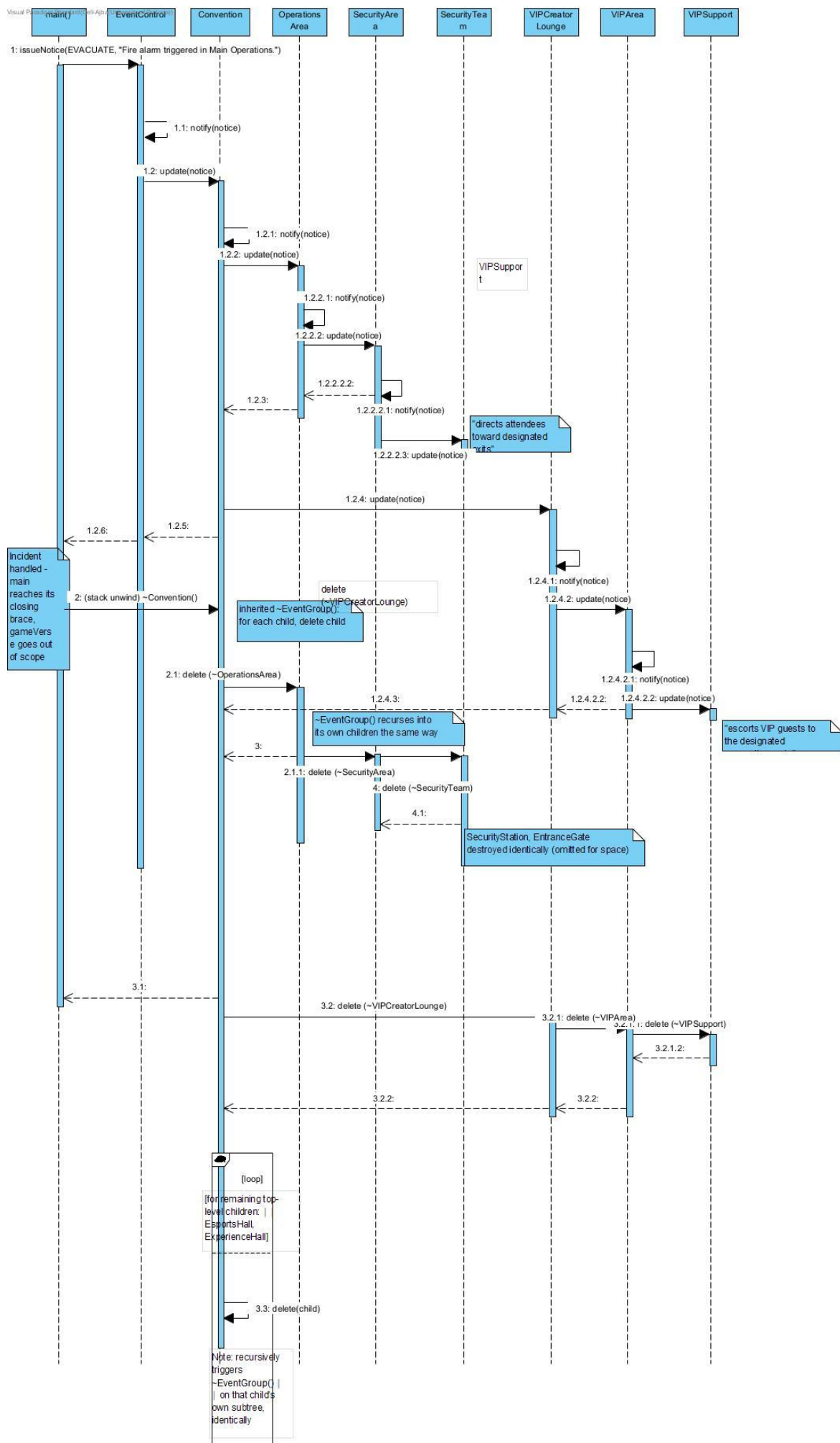
AnnouncementBoard is a new ConcreteObserver. It receives pushed notices and uses the notice's polymorphic getPublicMessage() to create an attendee-facing announcement. This gives one notice multiple independent outcomes: operational units change operations while the board communicates with attendees.

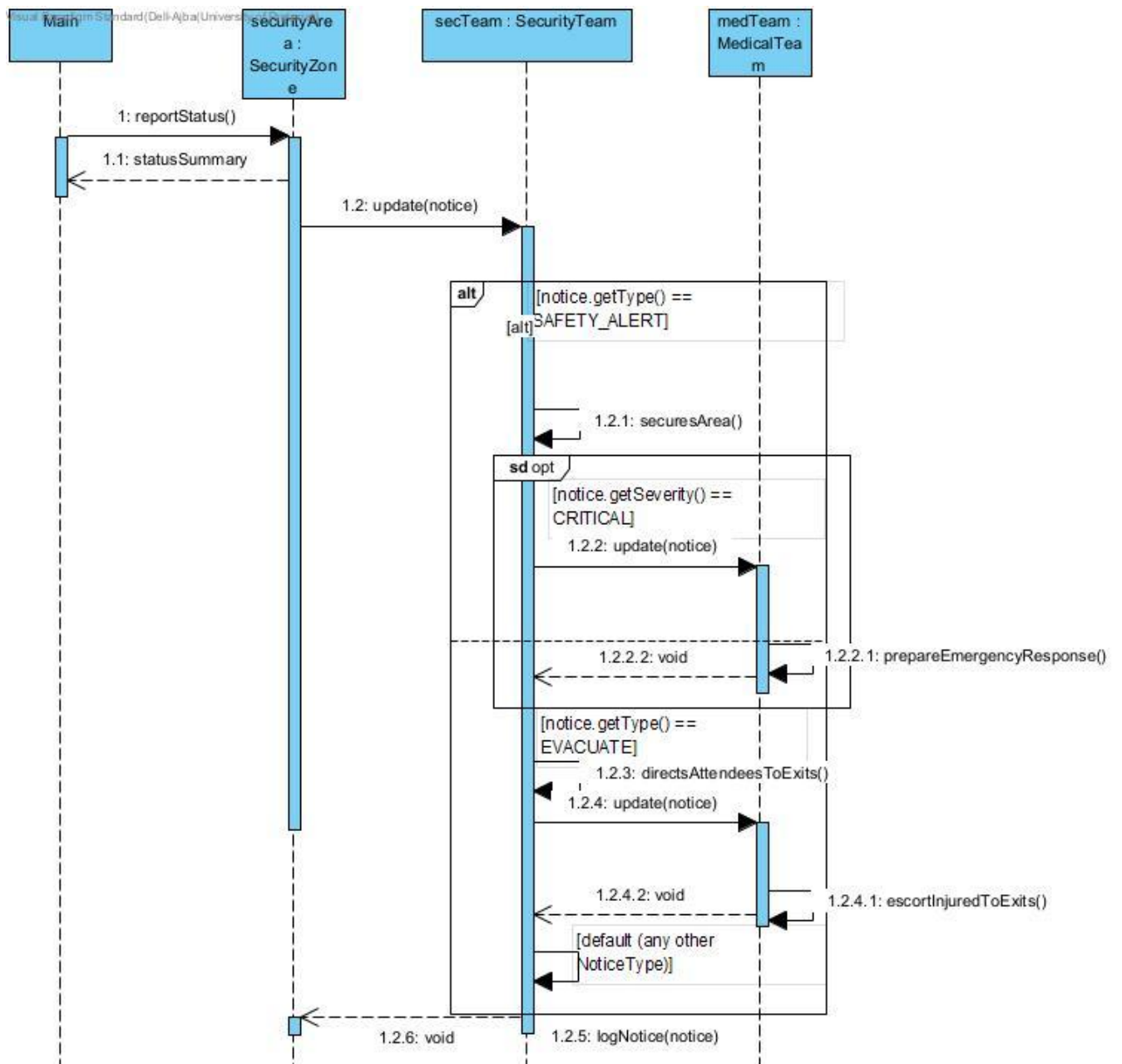
3. Dynamic Observer Subscription

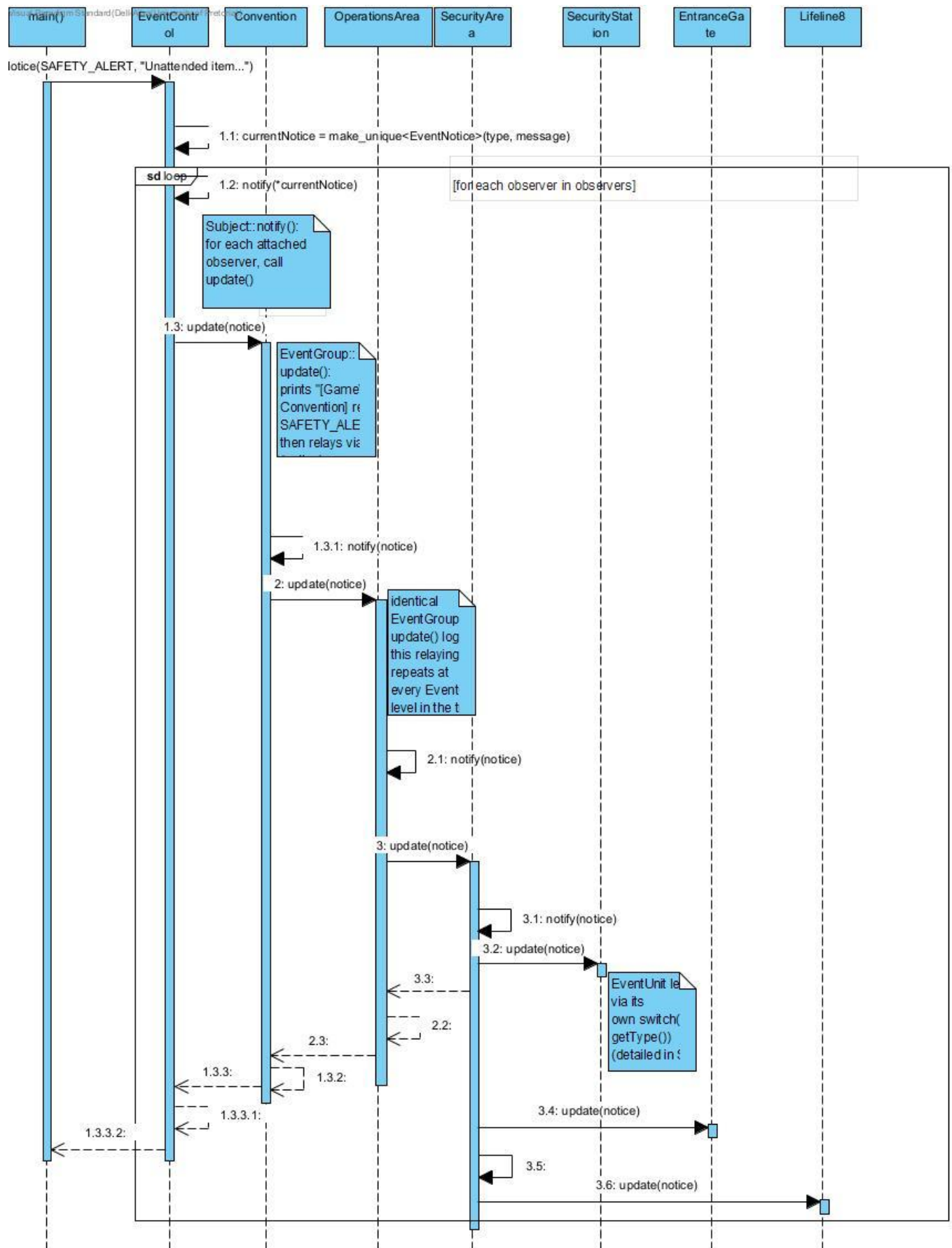
AnnouncementBoard can subscribeTo(Subject) and unsubscribeFrom(Subject) at runtime. The demonstration attaches it, attempts a duplicate attachment, detaches it so that it misses the next notice, and then re-attaches it. Subject::notify() uses a snapshot so registration changes during a notification affect the next notification rather than the current iteration.

TASK 5









TASK 7

7.4

GitHub Workflow Reflection

Our team divided work along GoF pattern lines from the start: Lesedi owned the Composite structure (event concept, five concrete leaf types, the composite tree, and destruction/ownership rules), Kiara owned the Observer system (the completed UML class diagram, design rationale, and notice cascading), and Ajba owned integration — the four sequence diagrams, main.cpp, and documentation. Each member worked on a dedicated feature branch (lesedi, Kiara, ajba), merged into main via pull request rather than direct pushes.

The repository's commit history reflects this split directly: Lesedi's early commits (added files to create skeleton, fixed file skeleton, added pdf with answers to 1.1, 1.2 and 4.1) established the project structure and the written event-concept/rules answers, while Kiara's commits (Added design rationale, UML in progress, Updated UML, Added UML and Object diagram to pdf) built out the class diagram and design justification in parallel on a separate branch. Two pull requests — one opened by Kiara, one by Lesedi — brought each branch's work into main, with a subsequent Merge branch 'main' commit and a removed docs cleanup commit showing the team resolving overlap between the two streams of work rather than one person dumping a final version at the end.

Using separate branches per person meant structural work (Composite skeleton) and documentation work (UML/rationale) could proceed simultaneously without blocking each other, and pull requests gave each member visibility into what the others had changed before it landed on main.

Ajba (Person 3) is responsible for Task 5 (all four sequence diagrams), Task 7.1/7.4 (README + this reflection), and Task 8 (main.cpp, demo prep). Commits for this work will be made on the ajba branch and merged into main via pull request, consistent with Lesedi and Kiara's workflow.